

CS 2110 Timed Lab 4: GBA

Henry Bui, Shivani Vora, Willson Pan, Galadriel Cho,
Nathan Duggal, Owen Anderson, Michael Yi, Ronit Jain

Spring 2025

Contents

1	Timed Lab Rules - Please Read	2
2	Overview	2
2.1	Purpose	2
2.2	Tasks	2
2.3	Criteria	2
3	Before You Start	3
3.1	Files	3
4	Instructions	4
4.1	Structs and Global Variables	4
4.2	Writing <code>redFilter</code>	4
4.3	Writing <code>greenFilter</code>	4
4.4	Writing <code>blueFilter</code>	4
4.5	Writing <code>drawImage</code>	5
4.6	Completing <code>main</code>	5
4.7	Testing	5
5	Rubric and Grading	6
5.1	Autograder	6
5.2	Makefile	6
5.3	Grading	7
6	Deliverables	7

Please take the time to read the entire document before starting the assignment. It is your responsibility to follow the instructions and rules.

1 Timed Lab Rules - Please Read

You are allowed to submit this timed lab starting from the moment your assignment is released until your individual period is over. You have 75 minutes to complete the lab, unless you have accommodations that have already been discussed with your professor. Gradescope submissions will remain open for several days, but you are not allowed to submit after the lab period is over. **You are responsible for watching your own time. Submitting or resubmitting after your due date may constitute an honor code violation.**

If you have questions during the timed lab, you may ask the TAs for clarification in lab, though you are ultimately responsible for what you submit. The information provided in this Timed Lab document takes precedence. If you notice any conflicting information, please indicate it to your TAs.

The timed lab is open-resource. You may reference your previous homeworks, class notes, etc., but your work must be your own. Contact in any form with any other person besides a TA is absolutely forbidden. **No collaboration is allowed for timed labs.**

2 Overview

2.1 Purpose

The purpose of this timed lab is to test your understanding of functions in a C program running on a Game Boy Advance (GBA) emulator.

2.2 Tasks

Complete the items marked as `TODO` in `main.c` using C. Detailed instructions for each function are on the following pages. First, you will implement the GBA helper methods `redFilter`, `greenFilter`, `blueFilter`, and `drawImage`. After, you will finish the `main` method to draw the background and filtered images (the images within the `image_array`) to the screen. Your program must use DMA to draw the images to the screen.

2.3 Criteria

Your assignment will be graded based on your ability to accurately implement the filters and draw the images in `image_array` to the specified locations on the screen. You will be graded on your use of DMA as well.

3 Before You Start

3.1 Files

The files we have provided:

- A `main.c` file that contains an outline for you to fill in!
- A `gba.h` file that contains all of the necessary GBA declarations such as `DMA`, `videoBuffer`, etc.
- A Makefile which provides commands you can use to compile and run your program—most importantly, `make mgba` to compile and run the script in the Docker terminal.
- The mGBA emulator (`mGBA.app`, `mGBA`, or `mGBA.exe`), `GBAServer.jar` and `client.c`. **DO NOT TOUCH THESE FILES.**
 - Note: If you are using MacOS you may have to manually open `mGBA.app` in your Finder before the command `make mgba` will work.

Feel free to use code from class resources as you need to, your own homeworks, but as always, not from your friends or AI!

4 Instructions

In this timed lab, the only file that needs to be edited is `main.c`. The `images` directory contains the images that you will filter and draw to the screen. Within `main.c`, you will implement the helper methods `redFilter`, `greenFilter`, `blueFilter`, and `drawImage`. You will also need to complete the main method to draw the filtered images to the screen.

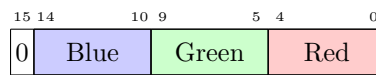
4.1 Structs and Global Variables

This timed lab involves the struct `ImgData`. `ImgData` represents the data needed to draw an image to the screen. The pointer `image` holds the address of the image data (in other words, its pixels). `width` and `height` are the image dimensions. `row` and `col` specify the location of the screen where the image should be drawn.

The global variable `videoBuffer` is a pointer to the GBA video buffer. The other global variables `background`, `puppy`, `puppy_red`, `puppy_green`, and `puppy_blue` contain the pixel data for the images. You do not need to access these.

4.2 Writing `redFilter`

Given a singular pixel value, this function should apply a red color filter onto it. To apply the red filter, zero out the other color channels while maintaining the bits of the red channel. As seen below, the red channel is comprised of bits 0 through 4:

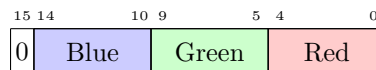


This function should return the newly filtered pixel, preserving only the bits within the red color channel. It will be called within the `main` method.

Recommendation: Write your bitmasks using hexadecimal (e.g. `0xFFFF`).

4.3 Writing `greenFilter`

Given a singular pixel value, this function should apply a green color filter onto it. To apply the green filter, zero out the other color channels while maintaining the bits of the green channel. As seen below, the green channel is comprised of bits 5 through 9:

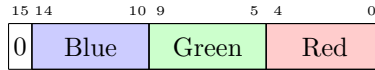


This function should return the newly filtered pixel, preserving only the bits within the green color channel. It will be called within the `main` method.

Recommendation: Write your bitmasks using hexadecimal (e.g. `0xFFFF`).

4.4 Writing `blueFilter`

Given a singular pixel value, this function should apply a blue color filter onto it. To apply the blue filter, zero out the other color channels while maintaining the bits of the blue channel. As seen below, the blue channel is comprised of bits 10 through 14:



This function should return the newly filtered pixel, preserving only the bits within the blue color channel. It will be called within the main method.

Recommendation: Write your bitmasks using hexadecimal (e.g. 0xFFFF).

4.5 Writing drawImage

This function must be implemented using DMA to draw an image at a specified row and column onto the screen. The parameters `width` and `height` represent the dimensions of the image. The pointer `image` holds the address. The parameters `row` and `col` specify the location on the screen to draw to.

Ensure that you are using the correct DMA channel. Useful macros and constants to assist drawing are contained in `gba.h`.

4.6 Completing main

Within `main`, we have already provided the code to apply the filters to the images. `image_array` has been created with `ImgData` structs that represent each image and how they should be drawn. The task left to you is drawing the images from `image_array` onto the screen.

Below the line marked `TODO`, implement the code to draw each of the images from `image_array` onto the screen. Each element of `image_array` is an `ImgData` struct containing the desired configuration of drawing to the screen. Use your helper method `drawImage` to accomplish this.

4.7 Testing

Within your terminal, run the Docker container. (The command is `./cs2110docker-gba.sh` for Linux and MacOS users and `./cs2110docker-gba.bat` for Windows users.) After that is set up, you may build your code and run the emulator with the following:

```
$ make mgba
```

If implemented correctly, you should see a simple screen of four upside down, differently colored labrador pictures in picture frames.

An example of a correct solution is below:



Figure 1: Expected Output

5 Rubric and Grading

5.1 Autograder

We have provided you with a test suite to check your work. You can run these using the Makefile.

Note: There are several object files `ag_utils.o`, `dma_framework.o`, and `dma_regions.o` that contain some functions that the test suite needs. We are not providing you the source code for this, so make sure not to accidentally delete this file as you will need to redownload the assignment. This file is not compiled with debugging symbols, so you will not be able to step into it with `gdb` (which will be discussed shortly).

The given test cases are the same as the ones on Gradescope. We formally reserve the right to change test cases or weighting after the lab period is over. However, if you pass all the tests, you can be confident that you will get 100% as long as you did not cheat or hard code in values.

Your code must not crash, run infinitely, nor generate memory leaks/errors.

If you need help with debugging, there is a C debugger called `gdb` that will help point out problems. See instructions in the Makefile section for running an individual test with `gdb`.

5.2 Makefile

We have provided a Makefile for this timed lab that will build your project. Here are the commands you should be using with this Makefile:

1. To clean your working directory (instead of manually deleting the `.o` files): `make clean`
2. To run your code and view the results in the GBA emulator: `make mgba`
3. To run all tests at once: `make run-tests`
 - To run a specific test: `make run-tests TEST=test_name`
 - E.g. `make run-tests TEST=test_drawImage::puppy`
4. To debug a specific test using `gdb`: `make run-tests-gdb TEST=test_name`

Then, at the (`gdb`) prompt:

- (a) Set some breakpoints (if you need to—for stepping through your code you would, but you wouldn’t if you just want to see where your code is segfaulting) with `b suites/suites.c:420`, or `b main.c:69`, or wherever you want to set a breakpoint
- (b) Run the test with `run`
- (c) If you set breakpoints: you can step line-by-line (including into function calls) with `s` or step over function calls with `n`
- (d) If your code segfaults, you can run `bt` to see a stack trace

To get an individual test name, you can look at the output produced by the tester. For example, the following failed test is `test_drawImage::puppy`:

```
[FAIL] test_drawImage::puppy: Timed out. (2.77s)
~~~~~
```

Beware that segfaulting tests will show the line number of the last test assertion made before the segfault, not the segfaulting line number itself. This is a limitation of the testing library we use. To see what line in your code (or in the tests) is segfaulting, follow the “To debug a specific test using `gdb`” instructions above.

5.3 Grading

- (30 points): Correct implementation of the `redFilter`, `greenFilter`, and `blueFilter` functions.
 - (15 points): Correct return values for each of the 3 functions run on a set of 5 distinct colors. (15 total tests cases, 1 point each)
 - (15 points): Correct return values for all three functions on all possible color values. (1 test case)
- (50 points): Correct implementation of the `drawImage` function.
 - (30 points): `drawImage` correctly writes an image to the video buffer. (1 test case)
 - (20 points): `drawImage` uses DMA to correctly write an image to the video buffer (1 test case)
- (20 points): Correct execution of the `main` function, utilizing all other functions to draw a background and 4 filtered puppy images to the video buffer. (1 test case)

6 Deliverables

Please upload the following files to Gradescope:

1. `main.c`

All non-compiling timed labs will receive a zero. If you want to avoid this, do not run gcc manually; use the Makefile as described below.